AI in Culture and Arts – Human-AI Interaction

Using Large Models

Dr. Benedikt Zönnchen
18th of July 2026

MUC.DAI
Munich Center for
Digital Sciences and AI

HM

myt Hochschule
für Musik und Theater
München

1. Pre-trained Models & Embeddings

2. Decoding Strategies

3. Transfer Learning

4. Low-Rank Adaptation (LoRA)

1. Pre-trained Models & Embeddings

2. Decoding Strategies

3. Transfer Learning

4. Low-Rank Adaptation (LoRA)

In the previous block we trained models **from scratch**:

1. Collect data (German folk songs)
2. Define an architecture (Transformer decoder)
3. Optimise all weights from random initialisation

Problem

Training from scratch requires large amounts of data and compute. Most applications cannot afford this.

Key Idea

Train a model *once* on a massive, general dataset. Then **adapt** it to the task at hand using far fewer resources.

This two-stage paradigm underlies almost every modern AI system:

1. **Pre-training:** learn general structure from billions of examples
   (done once, by a large organisation with significant compute)
2. **Adaptation:** specialise to the application at hand
   (done by you, on your smaller domain-specific dataset)

The pre-trained weights encode everything the model "knows." We do not throw this away, instead we **build on it**.

Pre-trained Models: What Do They Learn?

A model trained on enough text or music builds **internal representations** that capture:

1. **Structure:** grammar of language / grammar of melody
2. **Semantics:** meaning of words / character of musical phrases
3. **Style and register:** formal vs. informal / major vs. minor
4. **Long-range dependencies:** pronoun resolution / thematic development

These representations are stored in the model's **weight matrices**. Adaptation means *steering* these representations towards a new domain.

Definition

An **embedding** is a function $f : \mathcal{X} \to \mathbb{R}^d$ that maps discrete objects (words, sentences, notes, images) into a continuous vector space.

The central property: **similar objects map to nearby vectors**.

The Famous Example: Word2Vec (Mikolov, Chen, Corrado, & Dean, 2013)

$$f(\text{“king”}) - f(\text{“man”}) + f(\text{“woman”}) \approx f(\text{“queen”})$$

Arithmetic in the embedding space reflects *real-world relationships*.

Embeddings: Geometry of Meaning

In a well-trained embedding space:

1. **Distance** reflects semantic similarity
2. **Directions** encode relationships (gender, tense, genre, mood)
3. **Clusters** form for related concepts (instruments, keys, historical periods)

We measure similarity with **cosine similarity**:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|} \in [-1, 1]$$

1 = identical direction, 0 = orthogonal, -1 = opposite.

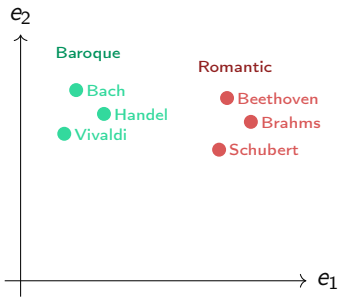


Figure: Schematic 2D projection of an embedding space.

Word embeddings assign one vector per word but meaning depends on **context**.

Sentence Transformers (Reimers & Gurevych, 2019)

A BERT-based encoder that maps an entire sentence to a single d-dimensional vector, trained so that **semantically similar sentences have high cosine similarity**.

The embedding is computed in three steps:

1. **Tokenise** the input text
2. Run through a **bidirectional transformer** (full context, not causal)
3. **Pool** the token outputs (e.g. mean of all positions) into one vector

Unlike our decoder-only transformer, the encoder sees *all* tokens at once. There is no causal mask.

Embeddings: Visualisation

Embedding vectors live in $\mathbb{R}^d$ with d often in the hundreds or thousands. We project to 2D for visualisation:

1. **PCA** (Principal Component Analysis): linear projection onto the two directions of maximum variance . It is fast, globally faithful
2. **t-SNE**: non-linear projection that preserves local neighbourhood structure. It is good for revealing clusters, harder to interpret globally

Interpretation

Clusters that align with human perception (same genre, same mood, same period) indicate that the model has learned **meaningful structure** from data.

Embeddings: Applications in Culture and Arts

Semantic Search

Embed a query and a document collection. Retrieve the items with the highest cosine similarity.

Cross-Modal Retrieval

"Find a painting that matches this musical mood." Embed both images and music in a shared space.

Similarity and Clustering

Group artworks, pieces, or texts by style or mood automatically.

Style Measurement

How similar is a generated piece to a reference corpus? Cosine similarity of embeddings gives a quantitative answer.

1. Pre-trained Models & Embeddings

2. Decoding Strategies

3. Transfer Learning

4. Low-Rank Adaptation (LoRA)

Recap: Language Model Output

At each step t, the model maps the context $x_1, \ldots, x_{t-1}$ to a vector of **logits** $\mathbf{l} \in \mathbb{R}^V$ (V = vocabulary size).
The probability distribution over the next token is:

$$p_i = \text{softmax}(\mathbf{l})_i = \frac{e^{l_i}}{\sum_{j=1}^{V} e^{l_j}}$$

Question

How do we **select** the next token from this distribution? The choice determines creativity, coherence, and diversity.

Greedy Decoding

Always choose the most probable token:

$$x_t = \arg\max_i \; p_i$$

Advantages

+ Deterministic and reproducible
+ Computationally cheapest
+ Locally coherent output

Disadvantages

- Highly repetitive output
- No diversity or creativity
- Globally suboptimal: locally safe choices compound

Temperature Sampling

Scale the logits by a temperature $T > 0$ before the softmax:

$$p_i^{(T)} = \frac{e^{l_i/T}}{\sum_{j=1}^{V} e^{l_j/T}}, \qquad \text{then sample } x_t \sim p^{(T)}$$

$T \to 0$	$T = 1$	$T \to \infty$
peaked distribution	model's own distribution	uniform distribution
predictable, safe	balanced	random, incoherent
(approaches greedy)		

Typical range for creative tasks: $T \in [0.7,\ 1.2]$.

Top-k Sampling

Restrict sampling to the k most probable tokens, renormalise, then sample:

$$V_k = \text{top-}k \text{ indices of } \mathbf{p}, \qquad p_i^{(k)} = \frac{p_i \cdot \mathbf{1}[i \in V_k]}{\sum_{j \in V_k} p_j}$$

This eliminates the long tail of very low-probability tokens (noise) while preserving diversity among the most plausible continuations.

Limitation

A fixed k is blind to context. When the model is confident, even the 2nd token may be very unlikely. When it is uncertain, $k = 50$ may still miss good candidates.

Top-p / Nucleus Sampling (Holtzman, Buys, Forbes, & Choi, 2019)

Select the *smallest* set V_p of tokens whose cumulative probability reaches p:

$$V_p = \min\left\{ S \subseteq \{1, \ldots, V\} \;\middle|\; \sum_{i \in S} p_i \geq p,\ S \text{ sorted by } p_i \text{ descending} \right\}$$

Renormalise over V_p and sample.

Unlike top-k, the nucleus size **adapts to the model's confidence**:

1. When the model is *certain*: $|V_p|$ is small (a few tokens carry most of the probability mass)
2. When the model is *uncertain*: $|V_p|$ is large (probability is spread across many plausible continuations)

	Greedy	**Temperature**	**Top-k**	**Top-p**
Deterministic	✓	–	–	–
Controls randomness	–	✓	✓	✓
Eliminates low-prob noise	✓	–	✓	✓
Adapts to confidence	–	–	–	✓
Useful for creative work	–	✓	✓	✓

In practice, temperature and top-p (or top-k) are often **combined**: scale the distribution with T, then apply nucleus sampling.

1. Pre-trained Models & Embeddings

2. Decoding Strategies

3. **Transfer Learning**

4. Low-Rank Adaptation (LoRA)

Scenario

We have a model pre-trained on a large general dataset. We want to adapt it to a new, different style but we have only a small dataset.

Transfer Learning

Initialise the model with the pre-trained weights (not randomly). Then continue training on the target data.

The pre-trained model already "knows" the general structure of the domain. We **transfer** this knowledge instead of starting from scratch.

The simplest approach is **full fine-tuning**: update all weights on the target data.

Problem: Catastrophic Forgetting

When *all* weights are updated on a small dataset, the model **overwrites its pre-trained knowledge**. After fine-tuning on Bach chorales, it may "forget" folk-song generation entirely.

Full fine-tuning on large models also has practical problems:

1. Expensive since gradients for every parameter must be stored
2. One full model copy per style thus storage cost grows linearly
3. Overfits easily on small datasets

Solution: Partial Freeze

Freeze the early layers (preserve general knowledge) and only **update the last few layers** (adapt to the new style).

The intuition:

1. **Early layers** learn low-level features (intervals, note durations) shared across styles
2. **Late layers** encode high-level stylistic choices that differ between styles
3. Updating only the last block and the output head strikes a balance between adaptation and preservation

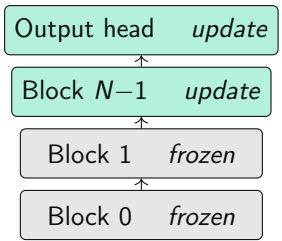


Figure: Partial freeze strategy.

Transfer Learning: Our Example

In notebook 10_3 we adapt our **folk-song Transformer** to **Bach chorales**:

1. **Pre-trained model:** Transformer decoder, 30 188 parameters, trained on 1 000 German folk songs for 120 epochs
2. **New data:** 43 Bach chorale soprano parts (music21 corpus)
3. **Strategy:** freeze all, update last transformer block + output head
   $\Rightarrow$ **46.6%** of parameters are trainable

Advantages

+ Works well with small datasets
+ Low risk of catastrophic forgetting
+ Fast training (fewer parameters updated)
+ Simple to implement

Disadvantages

- Coarse: entire layers are either fully frozen or fully updated
- Frozen layers *cannot* contribute to style adaptation at all
- Choosing which layers to freeze is a hyperparameter
- Still produces one full model copy per style

1. Pre-trained Models & Embeddings

2. Decoding Strategies

3. Transfer Learning

4. Low-Rank Adaptation (LoRA)

The Tension in Partial Freezing

Frozen layers cannot contribute to style adaptation at all. Unfreezing more layers risks overfitting on small datasets.

A Different Question

Instead of "*which layers do we update?*", ask:

"Can we add a small, trainable ***correction*** *alongside every weight matrix, while leaving the original weights permanently untouched?"*

Yes. This is the key idea behind **Low-Rank Adaptation (LoRA)** (Hu et al., 2021).

When adapting a model to a new task, the weight **change**

$$\Delta W = W_{\text{fine-tuned}} - W_{\text{pre-trained}}$$

tends to have **low intrinsic rank** meaning it lives in a much smaller subspace than the full matrix.

Low-Rank Approximation

If ΔW is low-rank, we decompose it as a product of two thin matrices:

$$\Delta W \approx BA, \quad B \in \mathbb{R}^{d_{\text{out}} \times r}, \quad A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad r \ll \min(d_{\text{in}}, d_{\text{out}})$$

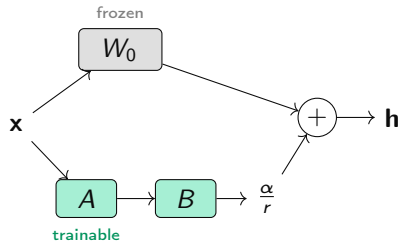
r is the **rank** hyperparameter — the key dial for the efficiency/capacity trade-off.

Forward Pass with LoRA Adapter

The original weight W_0 is **permanently frozen**. The adapter adds its contribution at every forward pass:

$$\mathbf{h} = \underbrace{W_0\,\mathbf{x}}_{\text{frozen}} + \underbrace{\frac{\alpha}{r}\,BA\,\mathbf{x}}_{\text{trainable adapter}}$$

where α is a scaling constant (often $\alpha = r$, giving scale $= 1$).



1. W_0 is **never modified**
2. Only A and B receive gradient updates
3. At deployment: merge $W_0 + \frac{\alpha}{r}BA$ into one matrix $\Rightarrow$ **zero inference overhead**

Zero-Start Property

- B is initialised to **zero**
- A is initialised with Kaiming uniform (random, non-zero)

Therefore $\Delta W = BA = 0$ at epoch 0. The adapted model produces **exactly the same output** as the pre-trained model at the start of training.

This matters because:

1. Training begins from a well-posed, meaningful starting point and not noise
2. There is no sudden destabilising shift at epoch 0
3. The adapter learns *incrementally* from the pre-trained equilibrium

LoRA: Parameter Savings

For a single weight matrix $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$:

Full fine-tuning	$d_{\text{out}} \times d_{\text{in}}$ parameters
LoRA rank r	$r\,(d_{\text{out}} + d_{\text{in}})$ parameters

Example: GPT-2 base ($d = 768$, 12 attention layers, Q/K/V targeted)

Strategy	Trainable params	% of model
Full fine-tuning (attn)	21 233 664	24.7%
LoRA $r = 8$	442 368	0.52%
LoRA $r = 2$	110 592	0.13%

At rank 8: a 48× reduction in trainable attention parameters.

Why the savings grow with model size

For a square layer of width d:

$$\frac{\text{LoRA params}}{\text{full params}} = \frac{r \cdot 2d}{d^2} = \frac{2r}{d}$$

As d grows, the fraction shrinks. For our tiny model ($d = 8$), $2r/d$ is large. For GPT-2 ($d = 768$, $r = 8$): 2%. For LLaMA 70B ($d = 8192$, $r = 8$): 0.2%.

LoRA's efficiency advantage grows with model size.

LoRA: Which Layers to Target?

LoRA is typically applied to the **attention weight matrices** because:

1. They carry most of the model's knowledge and stylistic "memory"
2. They are the largest linear layers in each transformer block
3. Query (W_Q), key (W_K), and value (W_V) projections are natural targets

In Our Transformer (notebook 10_4)

$3 \times 4 \times 2 = 24$ linear projections (key, query, value $\times$ 4 heads $\times$ 2 blocks). With rank $r = 4$: $24 \times 4 \times (32 + 8) = 3{,}840$ trainable parameters (**12.7%** of model) — all blocks adapted simultaneously.

LoRA: In Practice — the `peft` Library

For large models, the HuggingFace `peft` library (Mangrulkar et al., 2022) applies LoRA with a single call:

```
from peft import get_peft_model,
LoraConfig
model = get_peft_model(
  base_model,
  LoraConfig(
    r=8, lora_alpha=16,
    target_modules=["c_attn"],
    lora_dropout=0.05,
  )
)
model.print_trainable_parameters()
```

The library automatically:

1. Freezes all pre-trained weights
2. Adds A and B matrices alongside each target layer
3. Registers only A and B as trainable

The adapter is saved **separately**: $\sim$2 MB vs. $\sim$500 MB for GPT-2.

Why LoRA has become so dominant

- **Parameter-efficient:** trains $< 1\%$ of parameters at scale
- **No forgetting:** W_0 is never touched — pre-trained knowledge is preserved
- **Composable:** adapters for different styles can be swapped at runtime or linearly combined
- **Cheap to store:** one tiny adapter file per style, not one full model copy
- **Zero inference overhead:** merge $W_0 + \frac{\alpha}{r} BA$ before deployment
- **Enables consumer hardware:** fine-tune 70B-parameter models on a single GPU

Where LoRA Struggles

- **Rank is a hyperparameter:** too low $\Rightarrow$ underfits the style shift; too high $\Rightarrow$ loses parameter efficiency
- **Low-rank hypothesis may not hold:** some domain shifts require a high-rank update that LoRA cannot represent compactly
- **Choice of target layers matters:** targeting the wrong or too few layers limits adaptation
- **Modest gains on tiny models:** for our 30K-parameter model the savings are far less dramatic than for GPT-2
- **Still needs data:** a handful of examples is not enough for a dramatic style shift, especially on small models with limited capacity

LoRA: Applications

Language

1. Instruction-following adapters for LLaMA, Mistral, GPT
2. Domain-specific specialisation (clinical, legal, code)
3. Multilingual adapters
4. Persona and tone fine-tuning

Images

1. Stable Diffusion LoRA: 4 MB files for a specific artist style
2. Character or object adapters shared on the web

Music & Audio

1. Genre adapters for symbolic music generation models
2. Speaker adaptation for text-to-speech
3. Style transfer for melody generation

In This Course

1. Folk-song $\rightarrow$ Bach chorales (notebook 10_4)
2. GPT-2 $\rightarrow$ Shakespeare (bonus section)

LoRA: Comparison of Fine-Tuning Strategies

	Full FT	**Partial freeze**	**LoRA (small)**	**LoRA (large)**
Trainable params	100%	$\sim$47%	$\sim$13%	$< 1\%$
Layers touched	all	last block only	all (adapters)	all (adapters)
Forgetting risk	high	low	very low	very low
Adapter file size	full	full	small	tiny
Consumer GPU (LLM)	$\times$	$\times$	✓	✓

Full FT = full fine-tuning; Partial freeze = notebook 10_3; LoRA (small) = notebook 10_4 on our 30K model; LoRA (large) = peft on GPT-2 or similar.

Any questions?

References I

Holtzman, A., Buys, J., Forbes, M., & Choi, Y. (2019). The curious case of neural text degeneration. *CoRR*, *abs/1904.09751*. Retrieved from http://arxiv.org/abs/1904.09751

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., & Chen, W. (2021). LoRA: Low-rank adaptation of large language models. *CoRR*, *abs/2106.09685*. Retrieved from https://arxiv.org/abs/2106.09685

Mangrulkar, S., Gugger, S., Debut, L., Belkada, Y., Paul, S., Bossan, B., & Tietz, M. (2022). *PEFT: State-of-the-art parameter-efficient fine-tuning methods.* https://github.com/huggingface/peft.

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient estimation of word representations in vector space.* Retrieved from https://arxiv.org/abs/1301.3781

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using siamese BERT-networks. *CoRR*, *abs/1908.10084*. Retrieved from http://arxiv.org/abs/1908.10084